

Bitcoin Staking & Esbee DAO

Pooled Protocol Bonds on Stacks

A 30-Minute Technical Talk

Friedger — September 2026

Agenda

1. **Bitcoin Staking on Stacks** — What PoX-5 is and why it matters
2. **Protocol Bonds** — The dual-asset commitment mechanism
3. **The Bond Staker** — Pooled participation, epochs, and rolls
4. **The Bridge** — L1 bitcoin deposits with on-chain proof
5. **Leaving the Pool** — Three exit paths
6. **The Operator** — Five powers, and their limits
7. **Esbee DAO** — Quadratic governance for bond pools
8. **Testing & Verification** — Four layers of evidence
9. **In Closing** — What the system guarantees

Part 1: Bitcoin Staking on Stacks

What PoX-5 introduced, and where the yield comes from.

The Problem

Bitcoin earns nothing sitting still. Most yield means surrendering custody.

PoX-5: The Solution

Activated at Bitcoin block **960,230** (Epoch 4.0). The **genesis bond starts earning at block 966,350**.

- **Bitcoin Staking** — BTC-denominated yield without surrendering custody
- **Reward waterfall** — Paid from PoX rewards: miner revenue already spent competing for Stacks blocks
- **Dual-asset bonds** — A BTC leg and an STX lock, cryptographically linked
- **sBTC rewards** — Paid weekly via the sBTC auto-bridge, claimable as sBTC or Bitcoin L1

Part 2: The Protocol Bond

The dual-asset instrument the whole system is built on.

PART 2: THE PROTOCOL BOND

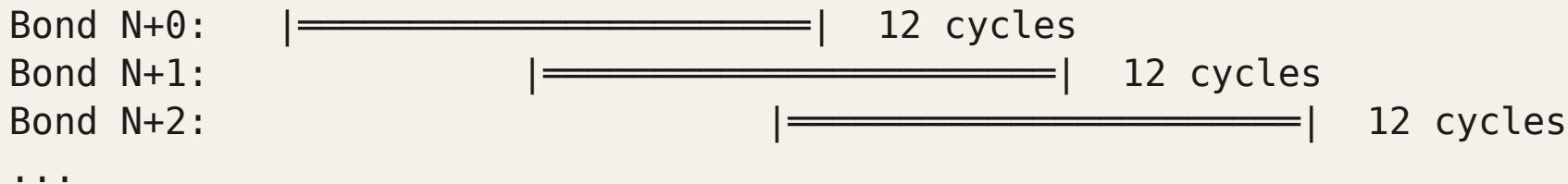
Anatomy of a Bond

A six-month, dual-asset commitment. The BTC leg has **two forms**:

PROTOCOL BOND	
BTC leg – either form	STX leg (Stacks)
(a) L1 timelock, own keys script commits to a hash of the Stacks principal	Locked in pox-5 Gates participation Determines signing weight
(b) sBTC lock on Stacks link encoded in Clarity	12 cycles $\approx$ 6 months No yield while paired
← the pool uses this one	

pox-5 refuses to register a bond it cannot match to a confirmed L1 UTXO. For sBTC locks, the same link is enforced in Clarity.

Six Bonds Overlapping



Steady state: 6 bonds active at once.

New bond opens every 2 cycles ($\approx$ monthly).

Bond N's term ends at exactly the cycle bond N+6 begins.

One STX principal = one bond at a time.

Part 3: The Bond Staker

One pooled position on the bond, and the ledger behind it.

Why a Pool?

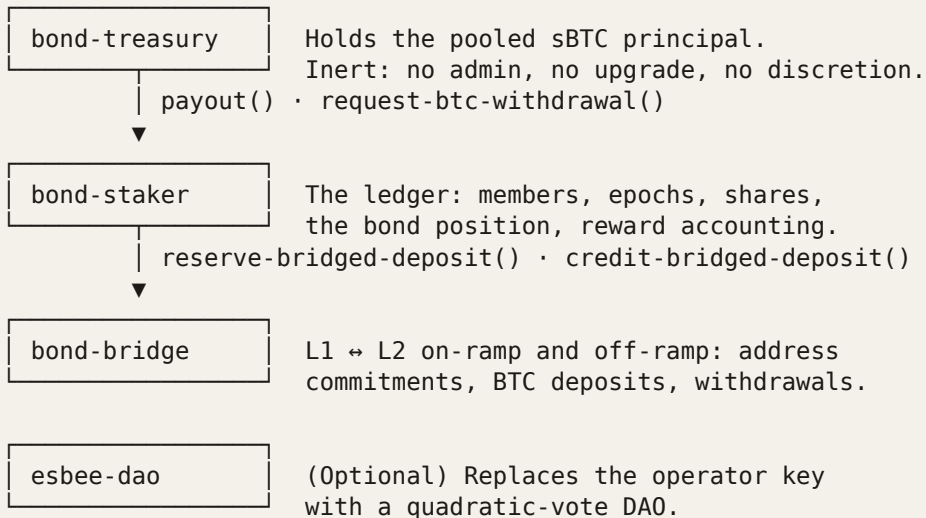
Not everyone has:

- A SegWit-capable Bitcoin wallet
- The STX balance for the protocol's STX:BTC ratio
- The infrastructure to monitor PoX-5 bonds

The pool aggregates these resources:

Member A: 50 sBTC + required STX ┐
Member B: 20 sBTC + required STX ├→ POOL CONTRACT → single pox-5 bond
Member C: 100 sBTC + required STX ┘
 |
 PoX-5 sees ONE staker
 Members tracked inside the pool

Four Contracts, One Purpose



Deploy order: treasury → staker → bridge → dao. Each calls only what is above it; there are no circular references.

Three Quantities Per Member

Quantity	Meaning
----------	---------

sats	sBTC deposited → what they get back
ustx	STX deposited → what they get back
shares	Weight in an epoch → determines reward share

Shares are struck when a bond is staked: **one share per committed satoshi**, mirroring PoX-5's own weighting. Tracked separately from deposits because the two quantities part ways depending on how a member leaves.

What Happens at a Roll

Each bond the pool stakes into is an **epoch**.

Current epoch (Bond N)

Members in bonded state

- |— Alice: 100 shares
- |— Bob: 50 shares
- |— Carol: 30 shares



stake() called

New epoch (Bond N+6)

Members in bonded state

- |— Alice: 100 shares
- |— Bob: 50 shares
- |— Dave: 20 shares (new!)

Carol: exited

Net sats difference → pox-5

STX resized in place

New epoch starts

The roll:

1. Adds queued deposits to the position
2. Releases members who **request-exit**
3. Carries remaining members forward **untouched**
4. Adjusts sBTC by the *net difference* only, resizing the STX lock in place

Two Clocks, One Stash

Rewards arrive as bare sBTC transfers, twice per reward cycle. An epoch's final cycle pays out *after* the roll that replaced it — so the pool runs two clocks:

Positions move at ROLL time:

- Member X enters Bond N at cycle 10

- Member X enters Bond N+6 at cycle 22

Rewards move at EPOCH SETTLEMENT time:

- Bond N's final cycle pays at cycle 26

- (half a cycle into Bond N+6)

STASH bridges the gap:

- When Bond N+6 is live but Bond N still pays,

- Member X's claim on Bond N rewards is set aside

- and draws down until that epoch settles.

Part 4: The Bridge

Joining and leaving with bitcoin on L1, proved on chain.

The Problem: sBTC Deposits Have No Sender

An sBTC deposit is a **bare mint** — signers credit the principal the tx named and call nothing. A deposit to the treasury arrives with **no record of who sent it**.

The bridge has members tie deposits to themselves *in advance*.

The Solution: Two-Phase Commit

Step 1: `commit-btc-address(digest, sats)`

- `SHA(address || salt)` committed on Stacks
- STX leg paid and held in the bridge
- Pool reserves allocation room

Step 2: `reveal-btc-address(address, salt)`

- Address revealed on Stacks
- `REVEAL_DELAY` = 2 burn blocks
- First reveal takes the address

Step 3: Member sends BTC from the revealed address

Step 4: sBTC signers sweep the deposit & mint to the treasury

Step 5: `complete-btc-deposit(txid, vout, tx, parents)`

- Permissionless. Proves every input locks to the address.

Why the Commit Protects You

Without commit:

- Alice reveals "bclq..." in mempool

- Bob sees it, races with a higher fee → takes Alice's deposit

With commit:

- Alice commits to $\text{SHA}(\text{address} \parallel \text{salt})$ → the digest says nothing

- Bob cannot guess the salt → cannot commit first

- Alice's reveal is mined first → takes the address

Why **two** burn blocks: **burn-block-height** moves per *tenure*, not per Stacks block. Two blocks guarantee at least one full Bitcoin block of margin — margin that fee competition cannot erase.

The Fast Lane: One-Call Claim

For addresses whose public key is recoverable (**p2pkh**, **p2sh-p2wpkh**, **p2wpkh**):

```
claim-btc-address(address, public_key, signature, sats)
```

Sign: "fastpool bond address claim: <contract_hash160>"
with your Bitcoin wallet's signmessage

Verifies:

- ✓ public key hashes to the address
- ✓ signature is valid for this contract
- ✓ no commit, no delay, no attack window –
a signature cannot be copied

Proving Deposit Ownership

At `complete-btc-deposit`, walk the tx chain:

sBTC registry names a txid it swept	← authenticated
tx must deserialize to that txid	← real deposit
each parent must deserialize to the txid its input names	← real outputs
output being spent → scriptPubKey	← locks to the
must match the revealed address	← revealed address

- **All** inputs must lock to the revealed address, not just one — a two-address transaction is then claimable by neither party alone
- Maximum **8 inputs** (each costs one parent tx)
- Uses Clarity 6's `get-bitcoin-tx-output?` — the node's own deserializer
- No merkle proofs, no block headers needed

Part 5: Leaving the Pool

Three exit paths, and what each one costs.

Three Exit Paths

Path	When	STX	sBTC	Rewards
request-exit	Released at next roll	Released at roll	Released at roll	Earned until roll
unstake-sbtc	After the bond's 12 cycles	Released	Released	Full term
unstake-sbtc-early	Any time	Stays locked in PoX-5	Released immediately	Forfeited

Early unstaking works the same **regardless of how a member joined** — native sBTC deposit or the L1 bitcoin bridge. Once deposited, every position is treated uniformly.

Early Exit Costs

unstake-sbtc-early costs:

- x STX leg stays locked in PoX-5 until the bond's normal unlock
- x Unrecognized rewards are forfeited
- x Future reward share on these sats is gone
- x PoX-5 drops the unstaked sats from the current and all future cycles

Does NOT cost:

- ✓ No member is diluted by someone else's exit
- ✓ No member subsidizes another
- ✓ The pool's reward stream shrinks by exactly the leaving shares

Preview before committing: `get-early-unstake-preview(member)`

Part 6: The Operator

Five powers, and the longer list of what they cannot reach.

Five Powers, Three Categories

Category	Power	What it does
Bond scheduling	set-next-bond	Places a floor on the next bond period – skip a bond or aim at one
Signing	update-bond-registration	Change the signer manager for the live bond
Signing	trust- / distrust-signer-manager	Add or remove a trusted signer code hash – a new one takes effect at the next roll, a distrusted one is removed instantly
Administration	update-operator	Add/remove operator keys (cannot modify own entry)
Administration	sweep-unattributed-principal	Move principal that arrived without a matching announcement

`update-bond-registration` is the most consequential: pox-5 pays rewards to the active signer manager, so a broken or malicious manager costs the pool an entire bond's worth of yield. Trust is tied to **code hashes**, so managers can be vetted before deployment.

What the Operator Cannot Do

- ✗ Touch deposits
- ✗ Prevent a bind — `bind-next-bond` is permissionless
- ✗ Prevent members from leaving
- ✗ Modify their own operator entry (no self-lockout)

Part 7: Esbee DAO

Replaces the deployer's operator key with on-chain governance. To start it, the sitting operator adds esbee-dao as an operator principal, then retires itself.

Quadratic Voting

Weight = $\sqrt{(\text{committed sats})}$

A holder 10,000× larger has 100× the say, not 10,000×.

Only **committed** shares count:

- Queued deposits are withdrawable on demand → deposit, vote, withdraw would rent a majority for the length of one transaction
- Committed sats are locked for the bond term → weight a voter must hold

Proposal Lifecycle

Every proposal must clear **all** of these lines:

Requirement	Value	Purpose
Voting period	~2 days (288 burn blocks)	Cannot be raised and settled before anyone reads it
Quorum	3,000 bips (30% of $\sqrt{\text{total-shares}}$)	An empty room does not decide
Supermajority	6,000 bips (60% of votes cast)	A bare majority is not a mandate
Execution delay	~1 day (144 burn blocks)	Members can request-exit before the outcome lands
Execution window	~1 week (1,008 burn blocks)	A stale mandate cannot be dusted off months later
Same epoch	Always	If the pool rolls, the membership that voted is not the one affected

Execution is **permissionless** — the mandate is the vote, not the executor. Proposing, voting and executing each emit a **print** an indexer can watch.

Six Typed Entry Points

<code>propose-trust-signer(code-hash)</code>	→ <code>execute-trust-signer(id)</code>
<code>propose-distrust-signer(code-hash)</code>	→ <code>execute-distrust-signer(id)</code>
<code>propose-signer-change(manager, old)</code>	→ <code>execute-signer-change(id, ...)</code>
<code>propose-operator-change(who, enabled)</code>	→ <code>execute-operator-change(id)</code>
<code>propose-next-bond(index)</code>	→ <code>execute-next-bond(id)</code>
<code>propose-sweep(recipient)</code>	→ <code>execute-sweep(id)</code>

Each proposal's kind is typed — a mandate for one power cannot be spent on another. Parameters are stored as plain fields, not an opaque payload, so what was voted on is legible on chain.

The DAO cannot touch deposits, and cannot remove itself from the operator seat.

Part 8: Testing & Verification

Four complementary layers, each catching a different class of defect.

Unit Tests

- ~**3,600 lines** of vitest across **six files**, covering every public entry point
- Run against **simnet**, which includes a **real pox-5 deployment** — bond admin, reward accounting and signer validation are exercised, not stubbed
- The fixture calls **setup-bond** and allowlists the pool through the boot address, so registration is verified against the protocol rather than mocked

Invariant Fuzzing (Rendezvous)

- **23 invariants**, living as `define-read-only` functions inside `bond-staker.clar` itself — Clarinet compiles twice, stripping `#[env(simnet)]` forms for the publish build
- pox-5's sBTC custody is stood in for by `bond-escrow.clar`, modelled as a **real token contract with actual transfers** — so balance invariants cannot hold trivially

Three real defects caught:

Defect	What went wrong
Rounding hole in rewards	<code>sync-rewards</code> credited <code>floor(a) + floor(b)</code> , which can be one sat short of <code>floor(a+b)</code> ; the final <code>claim-rewards</code> would abort on underflow
Principal conjured by rounding	A roll that could not carry all queued deposits took the returned part as the remainder, rounding it up — members collectively owed one sat more than credited
Double-refund on the settlement clock	Epochs stayed open for rewards past their roll and positions waited too, so withdraw refunded a deposit a second time out of another member's queued funds

Stxer Simulations

- End-to-end runs against a **fork of real mainnet state** — the actual deployed **pox-5** and **sBTC**, not stand-ins
- Steps sent from principals the developers hold no key for — the only way to model the bond admin's side of a launch
- Nothing signed or broadcast: deploy → allowlist → bind → deposit → stake
- Two scenarios: **simulate:genesis** (native sBTC) and **simulate:bridge** (L1 path)
- Each prints a per-step report and a stxer URL, so anyone can replay the run

Symbolic Verification (Clairvoyance)

Experimental — built on Jude Nelson's ongoing research into formal verification of Clarity.
Run against **bond-staker** with 18 invariants:

```
155  HOLDS
    0  VIOLATED
    2  NOT PROVEN    (known tool limitation around (or) expressions)
401  UNFINISHED    (engine step budget; settle's branching factor)
```

Mutation testing guards the result. Scripts break the contract on purpose and require the invariant they break to stop holding — every mutator that introduces a bug flips at least one invariant from HOLDS to NOT PROVEN. This guards against symbolic verification's primary failure mode: invariants whose own reads get abstracted away hold trivially, making a wall of HOLDS indistinguishable from correctness.

*This is **evidence, not a clean bill of health**. The UNFINISHED results are what the tool could not reach — not a verdict.*

Part 9: In Closing

Pooled Bitcoin staking at the protocol level — secure, permissionless, and governed by the people who stake.

Why This Matters

- 1. Permissionless exit guarantees** — `bind-next-bond`, `stake`, `unstake-sbtc` and both claim functions are permissionless. The operator can never strand the pool; members can always leave.
- 2. L1 security through on-chain proof** — Bitcoin transaction ancestry verified with Clarity 6's native `get-bitcoin-tx-output?`, anchored by the sBTC registry. No merkle proofs, no block headers.
- 3. Quadratic voting without identity** — $\sqrt{\cdot}$ -weighted voting over committed stake. No identity system, no per-voter credit budget.
- 4. Zero operator trust for user funds** — The operator binds bonds, picks signers and manages the skip, but can never touch deposits.
- 5. Composability with PoX-5** — The pool follows the protocol's own bond lifecycle and roll mechanism rather than building a parallel state machine.

Permissionless Guarantees

Function

Why It Matters

`bind-next-bond()`

Anyone can start the pool. No single point of failure.

`stake()`

Anyone can roll the bond. The operator cannot freeze progress.

`unstake-sbtc()`

Anyone can wind down the pool at term end.

`sync-rewards()`

Anyone can recognize sBTC rewards.

`claim-rewards() / claim-principal()`

Anyone can claim on behalf of any member.

`complete-btc-deposit()`

Anyone can finish a member's bridged deposit.

`execute-*(id)`

Anyone can execute a passed DAO proposal.

Q&A

Thank you.

Resources:

- Forum post: forum.stacks.org/t/bond-staking-and-esbee-dao/18972
- GitHub: [fastpool/sbtc-pool-bond-staker](https://github.com/fastpool/sbtc-pool-bond-staker)
- PoX-5 docs: docs.stacks.co
- License: CC0 1.0 Universal (Public Domain)